

University of Runnda
College of Science and Technology.
Nyanunge Campus.
Computer and software Engineering.

Reg: 223021200

Date: 27th Feb 2026

Module: Mobile application systems and Design

Assignment #1

Topic: State Management in Flutter.

Qn 1.

- Provider is a wrapper around inherited Widgets that makes them easier to use and more reusable. It is officially recommended approach for most developers due to its simplicity and balance of power.
- Riverpod (often called "Provider 2.0", it is a complete rewrite of provider that catches programming errors at compile-time rather than runtime. It doesn't depend on the Flutter BuildContext making it more flexible and testable.
- BLOC (Business Logic Component): A pattern that separates the presentation layer from the business logic. It relies on streams and sinks to handle events and states, enforcing a strict, predictable data flow.
- GetX: An extra-light powerful solution that combines high-performance state management, intelligent dependency injection and route management. It is known for its simplicity and for not requiring BuildContext to navigate or show snackbars.

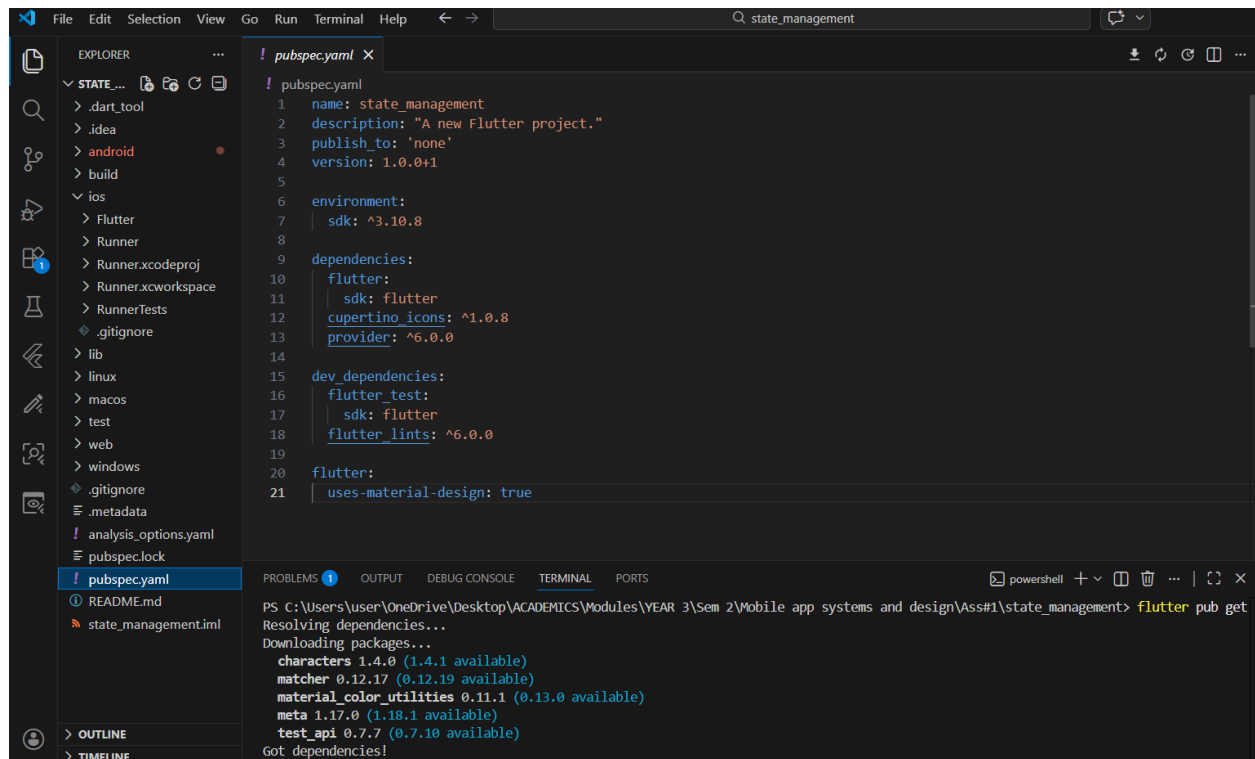
Qn2.

Situation	Provider	Riverpod	BLoC	GetX
Small applications	Very easy and simple to use.	Good but slightly advanced.	Too complex.	Very simple and fast.
Medium applications	Handle moderate complexity well.	More scalable and safe.	Good structure.	Fast and flexible.
Large/Enterprise applications.	Hard to manage when app grows.	Good for scalable apps.	Best choice for large apps.	Can become unorganized.
Team projects	Easy for teams to understand.	Supports clean collaboration.	Excellent for structured teams	Depends on discipline.
Fast development.	Quick to implement.	Moderate speed.	Slower due to setup.	Very fast development.
Strict Architecture requirement.	Basic structure only.	Stronger structure.	Very strict and organized.	Not strict.

Qn3.

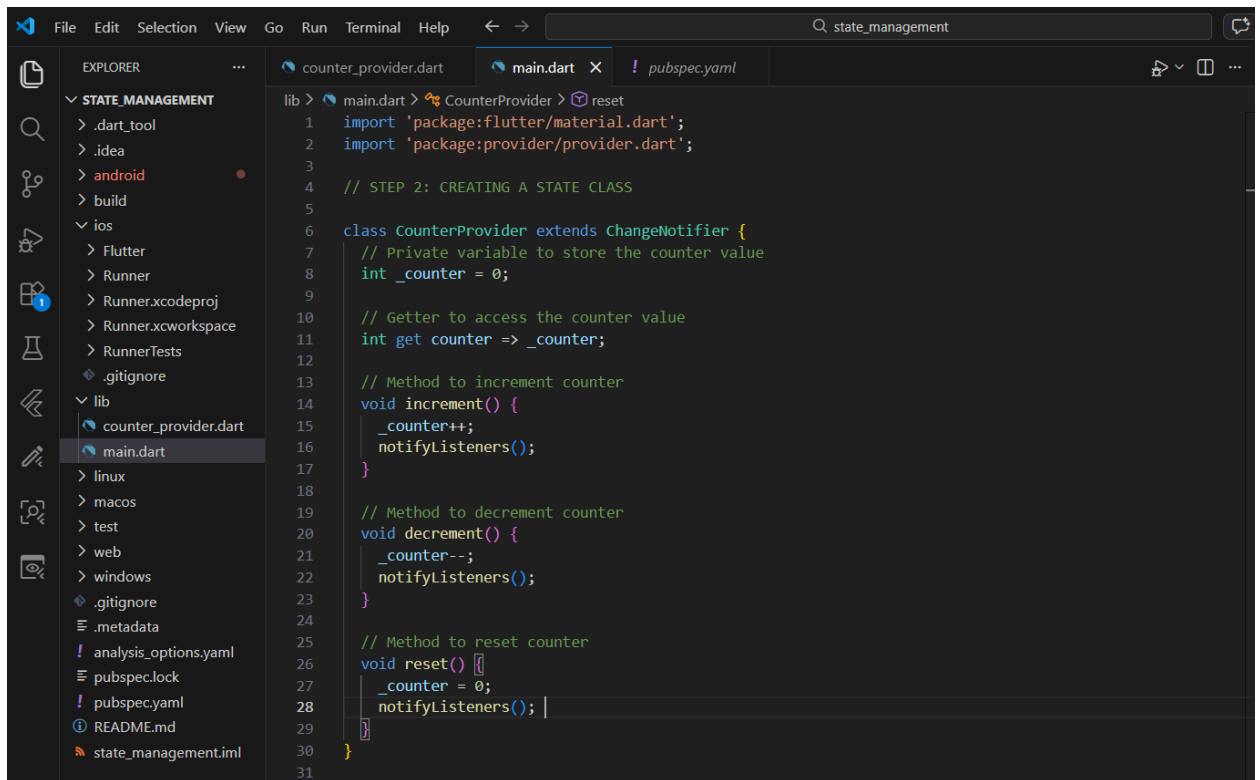
Q3.

- i. Adding dependency



- ✓ Dependencies go in [pubspec.yaml](#)
- ✓ Run: “flutter pub get” to download them

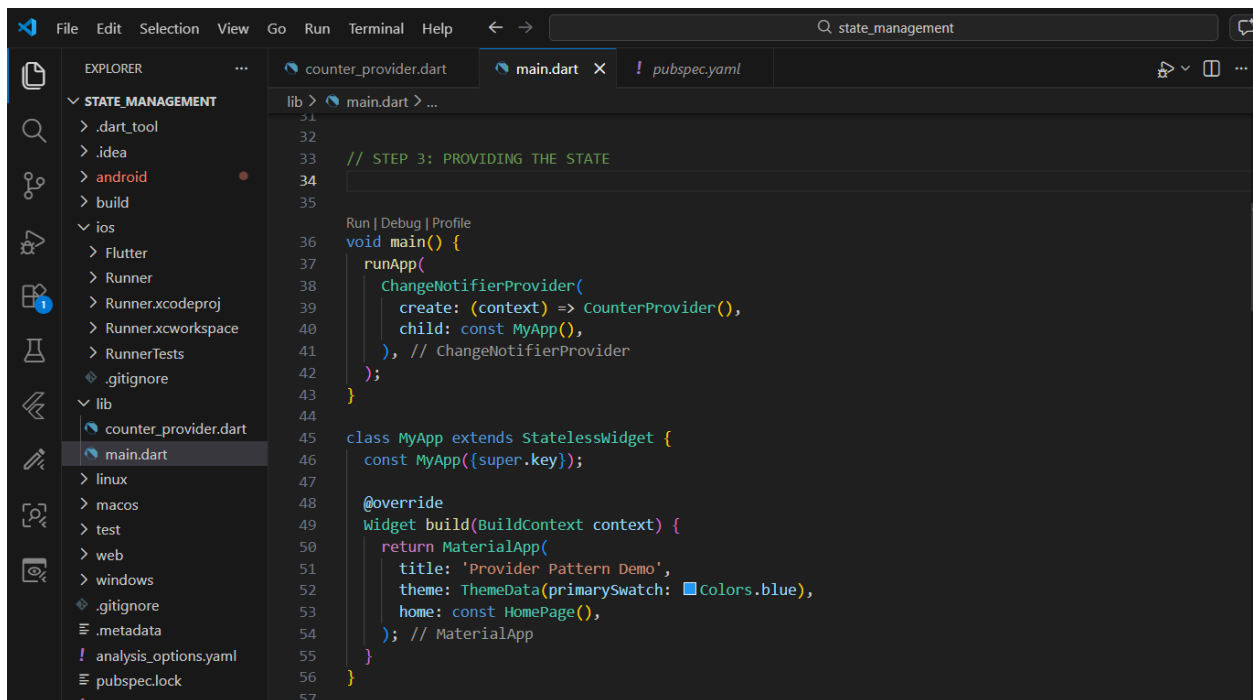
ii. Creating a state class



I created a new file called "counter_provider.dart" under lib folder

- A **ChangeNotifier** is a class that holds your app's state and notifies all listeners when the state changes.
- extends **ChangeNotifier** - makes this class observable
- **_counter** - private variable holding the state
- **notifyListeners()** - alerts all listeners that state changed

iii. Providing the state



```
lib > main.dart > ...
31
32
33 // STEP 3: PROVIDING THE STATE
34
35
Run | Debug | Profile
36 void main() {
37   runApp(
38     ChangeNotifierProvider(
39       create: (context) => CounterProvider(),
40       child: const MyApp(),
41     ), // ChangeNotifierProvider
42   );
43 }
44
45 class MyApp extends StatelessWidget {
46   const MyApp({super.key});
47
48   @override
49   Widget build(BuildContext context) {
50     return MaterialApp(
51       title: 'Provider Pattern Demo',
52       theme: ThemeData(primarySwatch: Colors.blue),
53       home: const HomePage(),
54     ); // MaterialApp
55   }
56 }
57
```

- **MyApp**: Wraps MaterialApp with MultiProvider to share CounterProvider across the app and **MaterialApp** sets app title and home page.

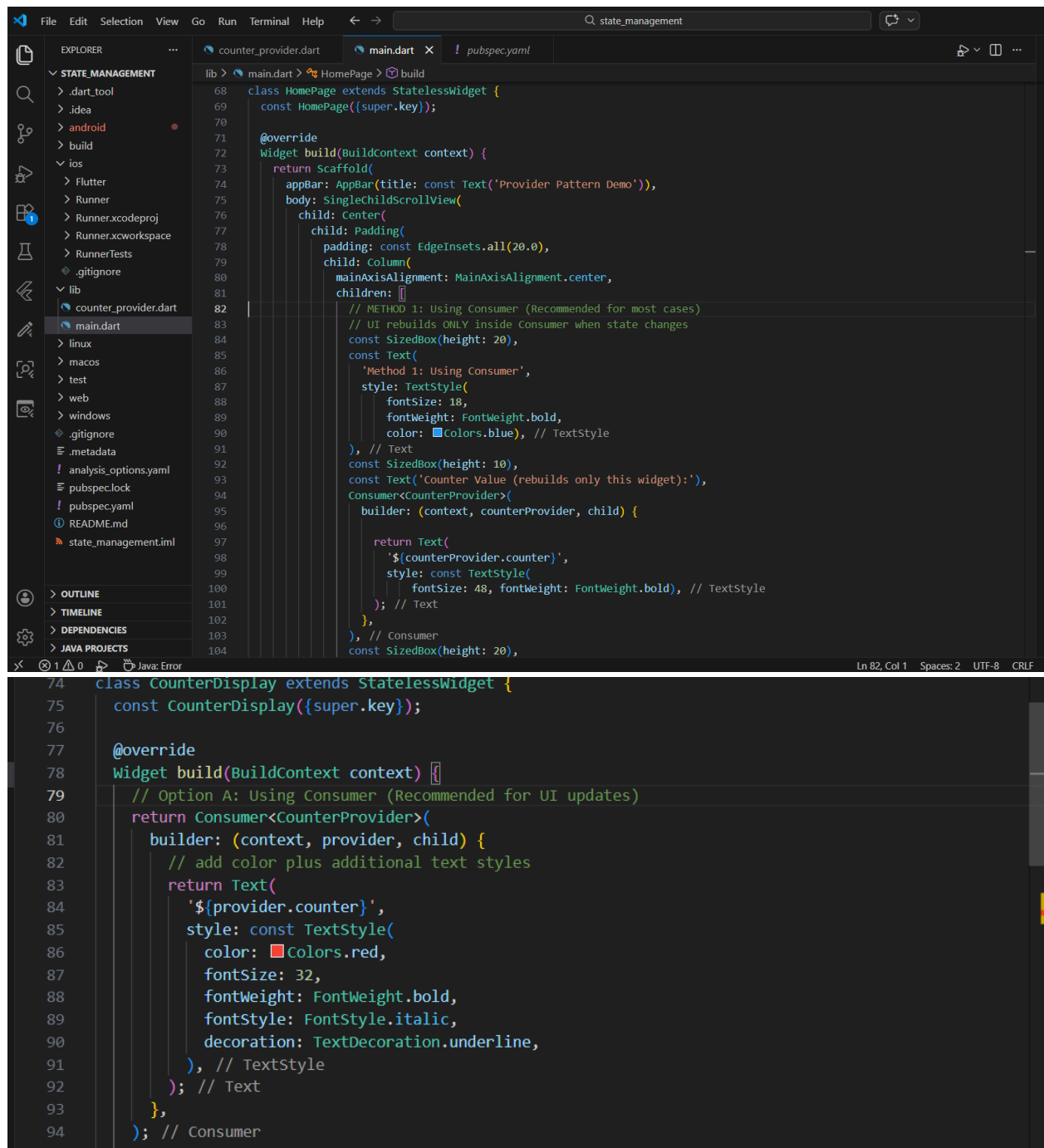
✓ This code Sets up a Flutter app with Provider state management for the counter.

Wraps the app with changeNotification to make counterProvider available to all widgets in the app.

- iv. Accessing the state
We have multiple ways to access the state:

Option A: **Using Consumer (Recommended for UI updates)**

Consumer (Recommended): Rebuilds only the widget inside the Consumer when state changes; best for performance.



```
68 class HomePage extends StatelessWidget {
69   const HomePage({super.key});
70
71   @override
72   Widget build(BuildContext context) {
73     return Scaffold(
74       appBar: AppBar(title: const Text('Provider Pattern Demo')),
75       body: SingleChildScrollView(
76         child: Center(
77           child: Padding(
78             padding: const EdgeInsets.all(20.0),
79             child: Column(
80               mainAxisAlignment: MainAxisAlignment.center,
81               children: [
82                 // METHOD 1: Using Consumer (Recommended for most cases)
83                 // UI rebuilds ONLY inside Consumer when state changes
84                 const SizedBox(height: 20),
85                 const Text(
86                   'Method 1: Using Consumer',
87                   style: TextStyle(
88                     fontSize: 18,
89                     fontWeight: FontWeight.bold,
90                     color: Colors.blue, // TextStyle
91                   ), // Text
92                 const SizedBox(height: 10),
93                 const Text('Counter Value (rebuilds only this widget):'),
94                 Consumer<CounterProvider>{
95                   builder: (context, counterProvider, child) {
96
97                     return Text(
98                       '${counterProvider.counter}',
99                       style: const TextStyle(
100                         fontSize: 48, fontWeight: FontWeight.bold, // TextStyle
101                       ), // Text
102                     ), // Consumer
103                   const SizedBox(height: 20),
104
```

```
74 class CounterDisplay extends StatelessWidget {
75   const CounterDisplay({super.key});
76
77   @override
78   Widget build(BuildContext context) {
79     // Option A: Using Consumer (Recommended for UI updates)
80     return Consumer<CounterProvider>(
81       builder: (context, provider, child) {
82         // add color plus additional text styles
83         return Text(
84           '${provider.counter}',
85           style: const TextStyle(
86             color: Colors.red,
87             fontSize: 32,
88             fontWeight: FontWeight.bold,
89             fontStyle: FontStyle.italic,
90             decoration: TextDecoration.underline,
91           ), // TextStyle
92         ); // Text
93       },
94     ); // Consumer
```

Option B: Using context.watch () (Newer, cleaner syntax)

It is the newer, cleaner syntax that also triggers rebuilds on change.

```

106
107 // METHOD 2: Using context.watch() (Recommended in functions)
108 // Allows you to access state value directly
109
110 const Text(
111   'Method 2: Using context.watch()',
112   style: TextStyle(
113     fontSize: 18,
114     fontWeight: FontWeight.bold,
115     color: Colors.green), // TextStyle
116 ), // Text
117 const SizedBox(height: 10),
118 const Text('Counter Value (using watch):'),
119 Text(
120   '${context.watch<CounterProvider>().counter}',
121   style: const TextStyle(
122     fontSize: 36, fontWeight: FontWeight.bold), // TextStyle
123 ), // Text
124 const SizedBox(height: 20),

```

Option C: Using Provider.of() (Traditional way)

It is the traditional approach, more verbose but still widely used.

```

125
126 // METHOD 3: Using Provider.of with listen: true
127 // Similar to watch but more traditional approach
128 const Text(
129   'Method 3: Using Provider.of(listen: true)',
130   style: TextStyle(
131     fontSize: 18,
132     fontWeight: FontWeight.bold,
133     color: Color.fromARGB(255, 63, 39, 3)), // TextStyle
134 ), // Text
135 const SizedBox(height: 10),
136 const Text('Counter Value (using Provider.of):'),
137 Text(
138   '${Provider.of<CounterProvider>(context, listen: true).counter}',
139   style: const TextStyle(
140     fontSize: 36, fontWeight: FontWeight.bold), // TextStyle
141 ), // Text
142 const SizedBox(height: 30),
143

```

v. Updating the state

Updating the State demonstrates how to call methods on the provider (like `increment()`) from UI elements such as buttons. When the method calls `notifyListeners()`, all listening widgets are notified and rebuild.

```
Go Run Terminal Help ← → state_management
counter_provider.dart main.dart ! pubspec.yaml
lib > main.dart > HomePage > build
68 class HomePage extends StatelessWidget {
72   Widget build(BuildContext context) {
143
144     // BUTTONS TO UPDATE STATE
145     // These buttons call methods on the provider using context.read()
146     // context.read() does NOT trigger rebuild - just accesses the value
147     const Text(
148       'Update State with Buttons:',
149       style: TextStyle(fontSize: 16, fontWeight: FontWeight.bold),
150     ), // Text
151     const SizedBox(height: 15),
152     Row(
153       mainAxisAlignment: MainAxisAlignment.center,
154       children: [
155         ElevatedButton.icon(
156           onPressed: () {
157             context.read<CounterProvider>().decrement();
158           },
159           icon: const Icon(Icons.remove),
160           label: const Text('Decrease'),
161           style: ElevatedButton.styleFrom(
162             backgroundColor: Colors.red,
163           ), // ElevatedButton.icon
164         const SizedBox(width: 10),
165         ElevatedButton.icon(
166           onPressed: () {
167             // Using context.read() to increment
168             context.read<CounterProvider>().increment();
169           },
170           icon: const Icon(Icons.add),
171           label: const Text('Increase'),
172           style: ElevatedButton.styleFrom(
173             backgroundColor: const Color.fromARGB(255, 114, 241, 118)),
174           ), // ElevatedButton.icon
175         const SizedBox(width: 10),
176         ElevatedButton.icon(
177           onPressed: () {
178             // Using context.read() to reset
179             context.read<CounterProvider>().reset();
180           },
181           icon: const Icon(Icons.refresh),
182           label: const Text('Reset'),
183           style: ElevatedButton.styleFrom(
184             backgroundColor: const Color.fromARGB(153, 255, 153, 0)),
185           ), // ElevatedButton.icon
186       ],
187     ), // Row
188     const SizedBox(height: 30),
189   },
190 }
```

vi. How UI rebuild happens

Explains the full flow: user action → provider method called → `notifyListeners()` triggers → listening widgets rebuild with the new state. This closes the loop on how Flutter's reactive UI works with Provider.

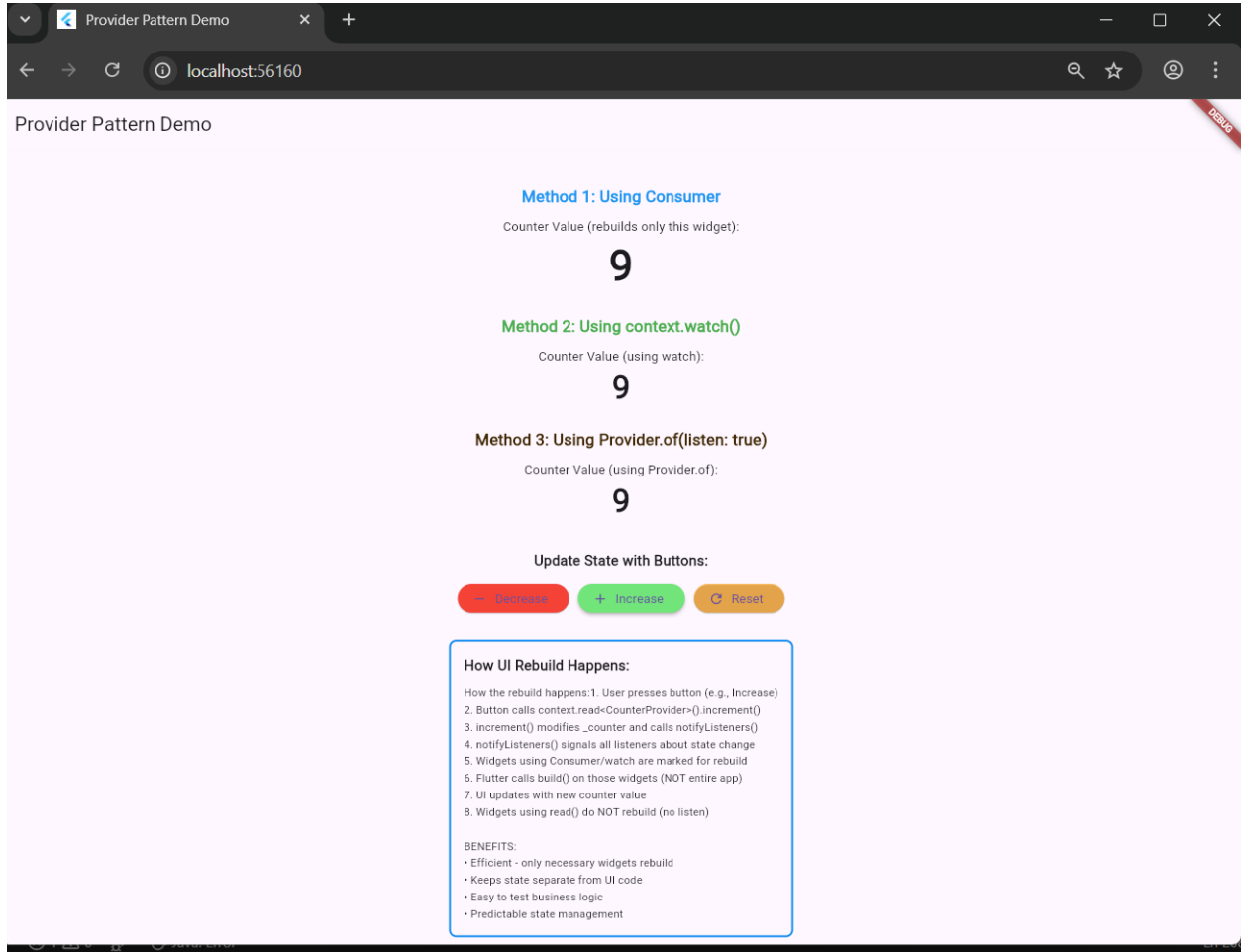

```

counter_provider.dart  main.dart  ! pubspec.yaml
lib > main.dart > HomePage > build
68 class HomePage extends StatelessWidget {
72   Widget build(BuildContext context) {
189
190     // =====
191     // UI REBUILD EXPLANATION
192     // =====
193     Container(
194       padding: const EdgeInsets.all(15),
195       decoration: BoxDecoration(
196         border: Border.all(color: const Color.fromARGB(255, 4, 133, 239), width: 2),
197         borderRadius: BorderRadius.circular(8),
198       ), // BoxDecoration
199       child: const Column(
200         crossAxisAlignment: CrossAxisAlignment.start,
201         children: [
202           Text(
203             'How UI Rebuild Happens:',
204             style: TextStyle(
205               fontSize: 16, fontWeight: FontWeight.bold), // TextStyle
206           ), // Text
207           SizedBox(height: 10),
208           Text(
209             'How the rebuild happens:'
210             '1. Click the button (Increase or Decrease)'
211             '2. Button calls context.read<CounterProvider>().increment()'
212             '3. increment() changes _counter and calls notifyListeners()'
213             '4. notifyListeners() announces "Counter changed!" to everyone.'
214             '5. Widgets using Consumer or .watch() receive the message'
215             '6. Their build() runs again with the new value'
216             '7. Screen updates to show the new number'
217             '8. Widgets using .read() stay the same (they do not listen)'
218             'Why this matters:'
219             '✓ Fast: Only important widgets rebuild, not everything'
220             '✓ Clean: State logic is separate from UI code'
221             '✓ Easy to test: No UI code mixed with data logic'
222             '✓ Predictable: You know exactly what causes updates'
223             '1. User presses button (e.g., Increase)\n'
224             '2. User presses button (e.g., Increase)\n'
225             '3. increment() modifies _counter and calls notifyListeners()\n'
226             '4. notifyListeners() signals all listeners about state change\n'
227             '5. Widgets using Consumer/watch are marked for rebuild\n'
228             '6. Flutter calls build() on those widgets (NOT entire app)\n'
229             '7. UI updates with new counter value\n'
230             '8. Widgets using read() do NOT rebuild (no listen)\n\n'
231             'BENEFITS:\n'
232             '• Efficient - only necessary widgets rebuild\n'
233             '• Keeps state separate from UI code\n'
234             '• Easy to test business logic\n'
235             '• Predictable state management',
236             style: TextStyle(fontSize: 12, height: 1.6),
237           ), // Text
238         ],
239       ), // Column
240     ), // Container
241     const SizedBox(height: 30),
242   ],
243 ), // Column
244 ), // Padding
245 ), // Center
246 ), // SingleChildScrollView
247 ); // Scaffold
248 }
249 }
250

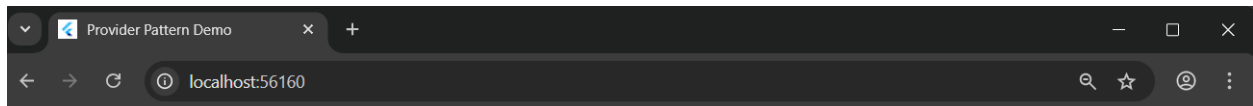
```

OUTPUT:

+increase



-decrease



Provider Pattern Demo

Method 1: Using Consumer

Counter Value (rebuilds only this widget):

4

Method 2: Using context.watch()

Counter Value (using watch):

4

Method 3: Using Provider.of(listen: true)

Counter Value (using Provider.of):

4

Update State with Buttons:



How UI Rebuild Happens:

How the rebuild happens: 1. User presses button (e.g., Increase)
2. Button calls `context.read<CounterProvider>().increment()`
3. `increment()` modifies `_counter` and calls `notifyListeners()`
4. `notifyListeners()` signals all listeners about state change
5. Widgets using `Consumer/watch` are marked for rebuild
6. Flutter calls `build()` on those widgets (NOT entire app)
7. UI updates with new counter value
8. Widgets using `read()` do NOT rebuild (no listen)

BENEFITS:

- Efficient - only necessary widgets rebuild
- Keeps state separate from UI code
- Easy to test business logic
- Predictable state management

Reset

Provider Pattern Demo

localhost:64740

Method 1: Using Consumer

Counter Value (rebuilds only this widget):

0

Method 2: Using context.watch()

Counter Value (using watch):

0

Method 3: Using Provider.of(listen: true)

Counter Value (using Provider.of):

0

Update State with Buttons:

- Decrease

+ Increase

Reset

How UI Rebuild Happens:

How the rebuild happens:1. User presses button (e.g., Increase)
2. Button calls context.read<CounterProvider>().increment()
3. increment() modifies _counter and calls notifyListeners()
4. notifyListeners() signals all listeners about state change
5. Widgets using Consumer/watch are marked for rebuild
6. Flutter calls build() on those widgets (NOT entire app)
7. UI updates with new counter value
8. Widgets using read() do NOT rebuild (no listen)

BENEFITS:

- Efficient - only necessary widgets rebuild
- Keeps state separate from UI code
- Easy to test business logic
- Predictable state management